Análise Comparativa da Efetividade de Testes Baseados em Propriedades em APIs REST Java/Spring Boot e Node.js/Express por meio de *Mutation Testing*

Miguel Vieira[1]

[1]Bacharelado em Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil
miguel.souza.1447219@sga.pucminas.br

Abstract

Property-Based Testing (PBT) checks general properties over automatically generated inputs, and Mutation Testing measures whether a test suite detects artificial faults. This work compares the effectiveness of equivalent PBT suites applied to two equivalent implementations of a REST API for event and ticket-reservation management, one in Java/Spring Boot (jqwik, PIT) and one in Node.js/Express (fast-check, StrykerJS). Eight properties derived from business rules were run 30 times per ecosystem. The mutation score was 50.0% in Java and 52.2% in Node.js; the difference is statistically significant (Mann-Whitney, $p < 0.001$) but small, and it vanishes for mutants of the same kind (30.8% vs. 29.2% for boundary mutants), because the two tools generate different operator mixes. Line coverage exceeded 91% in both ecosystems. Few surviving mutants were equivalent (3–4%); the rest fell in rules no property checks or escaped bound-only oracles. The model-based property was the most effective in both ecosystems.

Keywords: *Property-Based Testing. Mutation Testing. REST API. Java. Node.js.*

Resumo

Testes Baseados em Propriedades (PBT) verificam propriedades gerais sobre entradas geradas automaticamente, e o Mutation Testing *mede se uma suíte detecta defeitos artificiais. Este trabalho compara a efetividade de suítes PBT equivalentes aplicadas a duas implementações equivalentes de uma API REST de gestão de eventos e reservas de ingressos, uma em Java/Spring Boot (jqwik, PIT) e outra em Node.js/Express (fast-check, StrykerJS). Oito propriedades derivadas das regras de negócio foram executadas 30 vezes em cada ecossistema. O* mutation score *foi de 50,0% no Java e 52,2% no Node.js; a diferença é estatisticamente significativa (Mann-Whitney, $p < 0{,}001$), mas pequena, e desaparece para mutantes do mesmo tipo (30,8% contra 29,2% nos mutantes de limite), porque as duas ferramentas geram conjuntos de operadores diferentes. A cobertura de linhas passou de 91% nos dois ecossistemas. Poucos sobreviventes eram equivalentes (3% a 4%); os demais estavam em regras sem propriedade ou escaparam de oráculos que só verificam limites. A propriedade baseada em modelo foi a mais efetiva nos dois ecossistemas.*

Palavras-chave: *Testes Baseados em Propriedades.* Mutation Testing. *API REST. Java. Node.js.*

Bacharelado em Engenharia de Software – PUC Minas
Trabalho de Conclusão de Curso (TCC)
Orientadores de conteúdo (TCC I): Danilo Maia, João Pedro Batisteli e Leonardo Vilela
Orientador acadêmico (TCC I): Cleiton Tavares
Orientador do TCC II: [preencher]
Belo Horizonte, [data da entrega] de 2026.

1 Introdução

Uma suíte de testes pode executar quase todas as linhas de um sistema e, ainda assim, não perceber um comportamento incorreto. A cobertura de código mede o que foi executado, não o que foi verificado. Em APIs *backend*, esse descompasso pesa porque as regras de negócio combinam limites numéricos, estados e interações entre entidades, e testes baseados em exemplos verificam apenas os cenários que o desenvolvedor lembrou de escrever.

Testes Baseados em Propriedades (PBT, do inglês *Property-Based Testing*) atacam esse problema por outro ângulo: o desenvolvedor descreve uma propriedade geral do sistema e a ferramenta gera automaticamente centenas de entradas para tentar refutá-la [Claessen and Hughes, 2000]. Para medir se uma suíte de fato detecta defeitos, o *Mutation Testing* introduz pequenas alterações artificiais no código (mutantes) e verifica quantas delas fazem algum teste falhar; essa proporção é o *mutation score* [Jia and Harman, 2011; Papadakis et al., 2019].

A evidência disponível sobre a efetividade do PBT vem quase sempre de um único ecossistema. Ravi and Coblenz [2025] mediram por mutação as suítes de 40 projetos Python e mostraram que testes de propriedade encontram muito mais mutantes que testes unitários; Goldstein et al. [2024] e Corgozinho et al. [2023] estudaram como desenvolvedores escrevem propriedades, também dentro de um ecossistema. Ao mesmo tempo, ferramentas de mutação diferentes produzem conjuntos de mutantes diferentes para o mesmo programa [Kintis et al., 2018; Monteiro et al., 2022]. Não se sabe, portanto, se as mesmas propriedades, aplicadas a implementações equivalentes em ecossistemas distintos e avaliadas pelas ferramentas de mutação de cada um, levam ao mesmo resultado. Essa é a lacuna deste trabalho.

O estudo compara Java/Spring Boot e Node.js/Express, dois dos ecossistemas mais usados para construir APIs REST, por meio de duas implementações equivalentes de uma API de gestão de eventos e reservas de ingressos. As mesmas oito propriedades de negócio foram escritas com jqwik (Java) e fast-check (Node.js) e avaliadas com PIT e StrykerJS, respectivamente.

Questão de pesquisa geral: suítes de testes baseadas em propriedades, derivadas das mesmas regras de negócio, têm a mesma efetividade, medida por *mutation score*, quando aplicadas a implementações equivalentes de uma API REST em Java/Spring Boot e em Node.js/Express?

Objetivo geral: comparar, por meio do *mutation score*, a efetividade de suítes PBT equivalentes nas duas implementações. Os objetivos específicos, cada um ligado às questões de pesquisa (QP) da Seção 4.2, são:

- **OE1** caracterizar as propriedades derivadas das regras de negócio e sua equivalência entre jqwik e fast-check (QP1);
- **OE2** comparar o *mutation score* das duas suítes, inclusive sob o mesmo conjunto de categorias de operadores (QP2–QP4);
- **OE3** analisar a relação entre cobertura e *mutation score* em cada ecossistema (QP5);

OE4 classificar os mutantes sobreviventes segundo a regra de negócio afetada (QP6);

OE5 comparar o custo de execução e o esforço de configuração das ferramentas (QP7).

As contribuições são: (i) um experimento comparativo com 30 execuções por ecossistema e teste estatístico; (ii) a caracterização dos mutantes que sobrevivem a propriedades de invariante em cada ecossistema; e (iii) um repositório reprodutível com a especificação, as duas APIs, as suítes, os scripts de coleta e os dados brutos.

O restante do artigo está organizado assim: a Seção 2 apresenta os conceitos; a Seção 3, os trabalhos relacionados; a Seção 4, o método; a Seção 5, os resultados; a Seção 6, as ameaças à validade; e a Seção 7, as conclusões.

2 Fundamentação Teórica

2.1 Testes Baseados em Propriedades

Em PBT, o desenvolvedor escreve uma **propriedade**: uma afirmação que deve valer para todas as entradas de um domínio, como "o total de uma reserva nunca é negativo". Uma **invariante** é um caso particular, uma condição que toda operação deve preservar. A ferramenta usa **geradores** para produzir entradas aleatórias dentro do domínio e executa a propriedade para cada uma; quando encontra um contraexemplo, aplica ***shrinking***, reduzindo-o ao menor caso que ainda falha [Claessen and Hughes, 2000].

As bibliotecas atuais também injetam **casos de borda** (zero, mínimo, máximo, listas vazias) entre as entradas aleatórias. O jqwik, para a JVM, roda sobre a plataforma JUnit 5 [jqwik team, 2026]; o fast-check, para JavaScript e TypeScript, roda dentro de executores como o Jest [Dubien, 2026]. Ambos geram valores aleatórios com semente nova a cada execução, por padrão.

A qualidade de uma suíte PBT depende do que cada propriedade verifica. Uma propriedade que só checa limites ("$0 \leq$ desconto $\leq$ subtotal") aceita qualquer cálculo que respeite esses limites; uma que compara com um modelo de referência verifica valores exatos. Essa diferença de força é central na análise dos mutantes sobreviventes (Seção 5).

2.2 *Mutation Testing*

O *Mutation Testing* aplica **operadores de mutação** ao programa, cada um gerando uma versão com uma pequena alteração: trocar < por <=, + por −, remover uma chamada, devolver `null`. A suíte é executada contra cada mutante [Jia and Harman, 2011]. O mutante é **morto** quando algum teste falha, **sobrevivente** quando todos passam e **sem cobertura** quando nenhum teste executa o trecho alterado. Mutantes que estouram o tempo limite contam como detectados. O *mutation score* é:

$$MS = \frac{K + T}{K + T + S + N} \times 100\% \tag{1}$$

em que K, T, S e N são os mutantes mortos, com *timeout*, sobreviventes e sem cobertura. Mutantes que não compilam ou quebram a execução ficam de fora, como fazem PIT e

StrykerJS.

Um **mutante equivalente** altera o código sem alterar o comportamento observável e, por isso, nunca pode ser morto. Identificá-los exige análise manual [Papadakis et al., 2019]. Além disso, o conjunto de operadores varia entre ferramentas: o PIT muta *bytecode* Java [Coles et al., 2016] e seu conjunto padrão é mais restrito que o recomendado na literatura [Laurent et al., 2017]; o StrykerJS muta a árvore sintática de JavaScript e inclui operadores sem equivalente no PIT, como a troca de literais de texto [Stryker Mutator, 2026]. Comparar ecossistemas exige, portanto, comparar também por categorias de operadores comuns.

3 Trabalhos Relacionados

Os trabalhos abaixo foram escolhidos por avaliarem empiricamente PBT, ferramentas de mutação ou a combinação de ambos. A Tabela 1 resume como cada um se posiciona em relação a este estudo.

Ravi and Coblenz [2025] analisaram 426 projetos Python que usam Hypothesis, definiram 12 categorias de propriedades e avaliaram por mutação as suítes de 40 projetos. Cada teste de propriedade encontrou, em média, cerca de 50 vezes mais mutações que um teste unitário, e 76% das mutações encontradas apareceram nas primeiras 20 entradas geradas. É o trabalho mais próximo deste, mas observa um único ecossistema.

Goldstein et al. [2024] entrevistaram 30 usuários experientes de PBT na Jane Street e descreveram as dificuldades de projetar boas propriedades e geradores. **Corgozinho et al. [2023]** mineraram projetos Python com Hypothesis e mostraram que poucos padrões de propriedade concentram a maior parte do uso. Ambos orientaram a escolha de propriedades de formatos comuns (limites, comutatividade, neutralidade e modelo de transições) neste estudo.

Aichernig and Schumi [2019] derivaram propriedades e geradores de modelos de regras de negócio para testar um serviço web industrial com FsCheck, mostrando que regras de negócio são uma base viável para propriedades de APIs. Aqui as regras vêm de uma especificação textual e não de um modelo formal.

Bartocci et al. [2023] propuseram o *Property-Based Mutation Testing*, que interpreta cada mutante em relação à propriedade que ele viola. Este trabalho adota essa leitura ao classificar sobreviventes por regra e por propriedade, embora use as ferramentas de propósito geral PIT e StrykerJS.

Monteiro et al. [2022] e **Kintis et al. [2018]** compararam ferramentas de mutação para Java e mostraram que a ferramenta altera o resultado: o PIT gerou mais mutantes que o Major, e a vantagem se inverteu ao excluir os possivelmente equivalentes [Monteiro et al., 2022]. **Laurent et al. [2017]** mostraram que o conjunto padrão do PIT é restrito. Esses resultados motivam a análise por categorias comuns de operadores.

Beller et al. [2021] e **Sánchez et al. [2024]** estudaram a adoção industrial do *Mutation Testing*: mutantes precisam ser acionáveis, e o esforço de configuração é uma barreira. Isso justifica medir o custo de execução e registrar o esforço de configuração (QP7).

Tabela 1. Posicionamento dos trabalhos relacionados.

Trabalho	Técnica e linguagem	PBT	Mutação	Compara ecossistemas
Ravi and Coblenz [2025]	PBT, Python (Hypothesis)	sim	sim	não
Goldstein et al. [2024]	PBT, OCaml (entrevistas)	sim	não	não
Corgozinho et al. [2023]	PBT, Python (mineração)	sim	não	não
Aichernig and Schumi [2019]	PBT de serviços web, C#	sim	não	não
Bartocci et al. [2023]	Mutação por propriedade, Simulink	parcial	sim	não
Monteiro et al. [2022]	PIT × Major, Java	não	sim	não
Kintis et al. [2018]	Quatro ferramentas, Java	não	sim	não
Este trabalho	PBT, Java (jqwik) e Node.js (fast-check)	sim	sim	sim

4 Materiais e Métodos

4.1 Tipo de pesquisa

Trata-se de um experimento controlado comparativo, com análise quantitativa (*mutation score*, contagens de mutantes, cobertura e tempo, em 30 execuções por ecossistema) e qualitativa (classificação manual dos mutantes sobreviventes e registro do esforço de configuração). O fator estudado é o ecossistema, com dois níveis: Java/Spring Boot com jqwik e PIT, e Node.js/Express com fast-check e StrykerJS. Como cada ecossistema usa sua própria ferramenta de mutação, o fator é o par "linguagem + ferramentas", e não a linguagem isolada; essa escolha é discutida na Seção 6.

4.2 Objetivos, questões e métricas (GQM)

O estudo segue o modelo *Goal-Question-Metric* [Basili et al., 1994]. O objetivo é: *analisar* suítes de testes baseadas em propriedades *com o propósito de* compará-las *com respeito à* efetividade medida por *mutation score do ponto de vista* do desenvolvedor *no contexto de* duas implementações equivalentes de uma API REST em Java/Spring Boot e Node.js/Express. A Tabela 2 desdobra esse objetivo em questões e métricas; a Tabela 3 define cada métrica.

4.3 Aplicação de referência

A aplicação é uma API REST de gestão de eventos e reservas de ingressos, inspirada no domínio de plataformas como a Ticketmaster Discovery API [Ticketmaster Developer, 2026]. A API externa não é o objeto do experimento, porque o *Mutation Testing* exige acesso ao código. As regras foram fixadas em uma especificação única (`SPEC.md`), escrita antes das implementações, com 20 regras numeradas. A Tabela 4 as resume por módulo.

Tabela 2. Desdobramento GQM: objetivos específicos, questões de pesquisa e métricas.

OE	Questão de pesquisa	Métricas
OE1	QP1: Quais regras de negócio puderam ser expressas como propriedades equivalentes nos dois ecossistemas?	M1, M2, M3
OE2	QP2: Qual o *mutation score* de cada suíte?	M4, M5
OE2	QP3: A diferença de *mutation score* entre ecossistemas é estatisticamente significativa, e de que tamanho?	M6
OE2	QP4: A diferença se mantém quando só as categorias de operadores comuns às duas ferramentas são consideradas?	M7
OE3	QP5: Os mutantes cobertos pela suíte são também detectados por ela?	M8, M9
OE4	QP6: Quais regras e propriedades concentram mutantes sobreviventes, e por quê?	M10, M11
OE5	QP7: Qual ecossistema exige mais tempo de execução e esforço de configuração?	M12, M13, M14

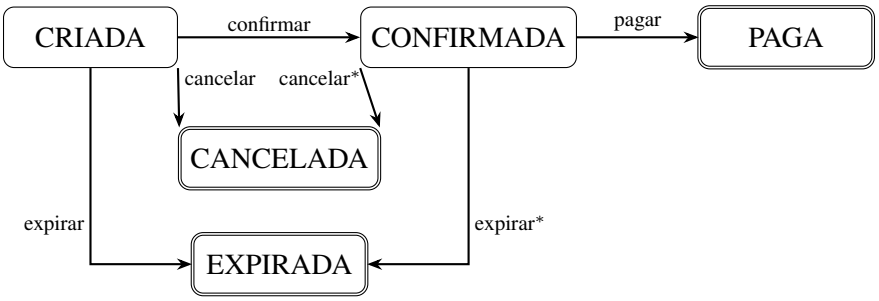


Figura 1. Transições de status da reserva (R18). Estados com borda dupla são finais (R19). *Devolve a disponibilidade dos lotes (R17).

4.4 Implementações e ferramentas

As duas implementações expõem as mesmas 14 rotas REST e delegam toda a regra a um serviço de domínio com persistência em memória. A Tabela 5 lista as versões. O código, a especificação, os scripts e os dados brutos estão no repositório do trabalho[1].

A equivalência funcional foi verificada por 11 cenários de exemplo compartilhados (`shared/casos-exemplo.json`), com 125 passos, dos quais 28 esperam um código de erro específico. O mesmo arquivo é lido por um executor em cada ecossistema, e os 11 cenários passam nas duas implementações. Esses testes de exemplo não entram na análise de mutação.

4.5 Propriedades

As oito propriedades previstas no TCC I foram implementadas nos dois ecossistemas com o mesmo enunciado, os mesmos geradores (mesmas faixas de valores) e 100 tentativas por propriedade (Tabela 6). Os geradores produzem de 1 a 4 lotes (preço de 0 a 100 000 centavos, quantidade de 1 a 50), de 1 a 5 itens (quantidade de 1 a 10) e cupons percentuais

[1] `https://github.com/[usuario]/tcc-pbt-mutation`

Tabela 3. Métricas: definição, forma de coleta e questão atendida.

Id	Métrica	Definição / fórmula	Fonte	QP
M1	Regras cobertas	Regras R1–R20 verificadas por ao menos uma propriedade / 20	`SPEC.md`	QP1
M2	Propriedades equivalentes	Propriedades de P1–P8 implementadas nos dois ecossistemas com o mesmo enunciado e os mesmos geradores	código	QP1
M3	Tamanho das suítes	Linhas de código não vazias e não comentário de propriedades e geradores	código	QP1, QP7
M4	Contagens de mutantes	Gerados, mortos, *timeout*, sobreviventes, sem cobertura e erros	PIT XML, Stryker JSON	QP2
M5	*Mutation score*	Equação 1, sobre todos os mutantes válidos	idem	QP2
M6	Diferença de *mutation score*	Mann-Whitney U, p-valor e $\hat{A}_{12}$ sobre os 30 valores por ecossistema	`runs.csv`	QP3
M7	*Score* por categoria	Equação 1 restrita às categorias de operadores comuns (Tabela 7)	`mutants.csv`	QP4
M8	Cobertura	Linhas e ramos do domínio cobertos pela suíte de propriedades	JaCoCo, Istanbul	QP5
M9	Cobertura de mutantes	Mutantes não classificados como "sem cobertura" / mutantes válidos, por módulo	`mutants.csv`	QP5
M10	Sobreviventes por recorte	Sobreviventes por módulo, categoria, e mortes por propriedade	`mutants.csv`	QP6
M11	Classe do sobrevivente	Classificação manual: equivalente, propriedade fraca ou regra sem propriedade	análise manual	QP6
M12	Tempo	Tempo de parede da suíte e da análise de mutação, por execução	`runs.csv`	QP7
M13	Mutantes por execução	Média de mutantes válidos gerados por execução	`runs.csv`	QP7
M14	Esforço de configuração	Arquivos de configuração, dependências e problemas registrados até a primeira execução	diário	QP7

Tabela 4. Módulos e regras de negócio da aplicação de referência.

Módulo	Regras	Conteúdo
Eventos	R1, R2	Capacidade > 0 e nome não vazio; só eventos `ATIVO` aceitam reservas, itens e confirmações.
Lotes	R3–R5	Preço ≥ 0 e quantidade > 0; soma das quantidades dos lotes $\leq$ capacidade do evento; $0 \leq$ disponível $\leq$ total.
Reservas	R6–R8	Reserva nasce `CRIADA`; itens só em reserva `CRIADA`, com quantidade > 0 e lote do mesmo evento; subtotal $= \sum$ preço $\times$ quantidade.
Cupons	R9–R13	Percentual em [1, 100] ou fixo > 0; desconto $= \lfloor$subtotal $\times$ $\%/100 \rfloor$ ou valor fixo, limitado ao teto; desconto zero se o cupom não existe, expirou ou o subtotal está abaixo do mínimo; $0 \leq$ desconto $\leq$ subtotal.
Disponibilidade	R14–R17	Confirmar exige itens e evento ativo; verifica todos os lotes antes de baixar qualquer um (tudo ou nada); cancelar ou expirar reserva confirmada devolve as quantidades.
Status	R18–R20	Transições da Figura 1; estados finais não mudam; entidade inexistente gera erro de "não encontrado".

Tabela 5. Ferramentas e versões usadas no experimento.

Papel	Java	Node.js
Plataforma	Java 17 (`-release 17`, JDK 21)	Node.js 22.22
Camada REST	Spring Boot 3.3.5	Express 5.3
Executor de testes	JUnit Platform 1.9	Jest 30.5
PBT	jqwik 1.7.4	fast-check 4.10.2
Mutação	PIT 1.18.2 + pitest-junit5-plugin 1.2.3, mutadores `DEFAULTS`	StrykerJS 10.0 + jest-runner, mutadores padrão
Cobertura	JaCoCo 0.8.12	Istanbul (via Jest)

ou fixos com mínimo, teto opcional (ausente em 25% dos casos) e validade de ± 30 dias.

4.6 Decisões de projeto do experimento

Para tornar a comparação justa e viável, foram tomadas as decisões abaixo, todas registradas no repositório:

- **Mutação só no domínio.** PIT e StrykerJS mutam apenas a camada de domínio (pacote `domain` e pasta `src/domain`); controladores e configuração do framework não contêm regra de negócio e só gerariam ruído.
- **Propriedades chamam o domínio diretamente**, não via HTTP, para que o tempo medido não inclua o custo de cada servidor web.
- **Dinheiro em centavos inteiros** nos dois lados, evitando divergências de ponto

Tabela 6. Propriedades implementadas, regras verificadas e tipo de oráculo.

Propriedade	Enunciado	Regras	Oráculo
P1 Total não negativo	Para qualquer reserva e cupom (válido, inexistente ou ausente), total ≥ 0	R8, R11–R13	limite
P2 Desconto limitado	$0 \leq$ desconto $\leq$ subtotal e total $=$ subtotal $-$ desconto	R11, R13	limite
P3 Comutatividade	Permutar os itens não altera o subtotal	R8	metamórfico
P4 Cupom inválido neutro	Cupom inexistente, expirado ou com mínimo acima do subtotal não altera o total	R12	neutralidade
P5 Disponibilidade não negativa	Após confirmações aceitas ou recusadas, $0 \leq$ disponível $\leq$ total	R5, R15	limite
P6 Redução correta	Confirmação aceita reduz exatamente o pedido; recusada não altera lotes	R15, R16	valor exato
P7 Cancelamento restaura	Confirmar e cancelar (ou expirar) devolve a disponibilidade inicial	R17	ida e volta
P8 Transições válidas	Sequências de 1 a 8 operações seguem o modelo da Figura 1 ou são recusadas sem mudar o status	R18, R19	modelo

flutuante que não são do ecossistema.

- **Só a suíte de propriedades** é usada na mutação e na cobertura; os testes de exemplo ficam de fora.
- **Configurações equivalentes:** 100 tentativas, geração aleatória e semente nova a cada execução nos dois lados. No jqwik, o reaproveitamento da semente que falhou na execução anterior foi desligado, para que um mutante não influencie o próximo.
- **Matriz completa de mortes:** o PIT registra todos os testes que matam cada mutante (*full mutation matrix*) e o StrykerJS foi configurado para não parar no primeiro teste que falha (`disableBail`), o que permite atribuir cada morte a uma ou mais propriedades.
- **Ferramentas como usadas na prática:** cada ferramenta de mutação roda com seu conjunto padrão de operadores. A comparação justa entre operadores é feita depois, por categorias (Tabela 7).

4.7 Procedimento e ambiente de execução

O experimento seguiu seis etapas (Figura 2). Cada uma das 30 execuções por ecossistema roda a suíte de propriedades com cobertura e, em seguida, a análise de mutação completa, gravando os relatórios brutos. Um script consolida os relatórios em duas tabelas: uma linha por execução (`runs.csv`) e uma linha por mutante por execução (`mutants.csv`). As execuções foram sequenciais (primeiro as 30 do Java, depois as 30 do Node.js), na mesma máquina virtual: 2 vCPUs Intel Xeon de 2,1 GHz, 7 GB de RAM, Ubuntu 24.04, com

Tabela 7. Categorias de operadores usadas para comparar as ferramentas. As cinco primeiras existem nas duas. †O grupo `DEFAULTS` do PIT 1.18 não inclui `NegateConditionals`; por isso, no Java, a categoria contém apenas mutantes de limite, enquanto o `EqualityOperator` do StrykerJS também nega comparações.

Categoria	PIT (`DEFAULTS`)	StrykerJS
Aritmético	`Math`, `InvertNegs`	`ArithmeticOperator`, `AssignmentOperator`, `UnaryOperator`
Relacional	`ConditionalsBoundary`†	`EqualityOperator`
Condicional/lógico	`RemoveConditional_*`	`ConditionalExpression`, `LogicalOperator`
Remoção de instrução	`VoidMethodCall`	`BlockStatement`, `CallExpression`
Incremento	`Increments`	`UpdateOperator`
Valor de retorno	`EmptyObject`, `Null`, `Primitive`, `True/FalseReturnVals`	—
Literal	—	`StringLiteral`, `BooleanLiteral`, `ObjectLiteral`, `ArrayDeclaration`
Chamada de método	—	`MethodExpression`, `OptionalChaining`

duas *threads* de mutação em cada ferramenta.

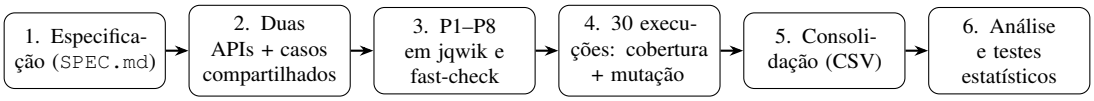


Figura 2. Etapas do procedimento experimental.

O ambiente de execução não tinha acesso ao repositório Maven Central. Por isso, jqwik 1.7.4 e PIT 1.18.2 foram compilados a partir do código-fonte das versões publicadas, e a suíte Java foi executada pelo JUnit Platform Console, com o PIT acionado pela mesma API que o *plugin* Maven usa. O `pom.xml` do repositório declara as mesmas versões, de modo que o experimento pode ser repetido com `mvn`. O esforço de configuração (M14) considera os dois caminhos.

4.8 Análise dos dados

O *mutation score* de cada execução forma uma amostra de 30 valores por ecossistema. Como o PBT é aleatório e não há garantia de normalidade, a diferença entre ecossistemas foi testada com Mann-Whitney U bilateral [Mann and Whitney, 1947] a $\alpha = 0{,}05$, e o tamanho do efeito foi medido pela estatística $\hat{A}_{12}$ de Vargha-Delaney [Vargha and Delaney, 2000], seguindo a recomendação de Arcuri and Briand [2011] para algoritmos aleatórios

em Engenharia de Software. $\hat{A}_{12}$ é a probabilidade de uma execução Java ter valor maior que uma Node.js; $0{,}5$ indica nenhuma diferença, e as faixas de magnitude são as de Vargha and Delaney [2000]. O teste de Shapiro-Wilk foi usado apenas para descrever a distribuição. As hipóteses são:

- $H_{0,1}$: o *mutation score* das suítes PBT não difere entre Java e Node.js; $H_{1,1}$: difere.
- $H_{0,2}$: o tempo da análise de mutação não difere entre os ecossistemas; $H_{1,2}$: difere.

Os mutantes sobreviventes foram agrupados por localização e operador e classificados manualmente em três classes: **equivalente** (nenhuma entrada distingue o mutante do original), **propriedade fraca** (a regra é exercitada por uma propriedade, mas o oráculo não observa a diferença) e **regra sem propriedade** (a regra afetada não é verificada por nenhuma de P1–P8).

5 Resultados

5.1 Sumarização dos resultados

Foram realizadas 60 execuções completas (30 por ecossistema), que somam 11 580 avaliações de mutantes sobre 386 mutantes distintos: 126 gerados pelo PIT no domínio Java e 260 gerados pelo StrykerJS no domínio Node.js. A Tabela 8 reúne os números principais. O resultado central é que as duas suítes de propriedades detectam cerca de metade dos mutantes, com *mutation score* médio de 50,0% no Java e 52,2% no Node.js, embora cubram mais de 91% das linhas do domínio.

Três números chamam atenção. Primeiro, o StrykerJS gerou o dobro de mutantes do PIT para um domínio menor em linhas de código, o que já indica que as ferramentas "enxergam" o código de formas diferentes. Segundo, o *mutation score* do Java foi idêntico nas 30 execuções, e no Node.js apenas 4 dos 260 mutantes mudaram de resultado entre execuções: a aleatoriedade do PBT quase não afetou o resultado. Terceiro, os 11 cenários de exemplo, escritos para verificar a equivalência entre as implementações e usados aqui apenas como linha de base, mataram mais de 92% dos mutantes nos dois ecossistemas, contra cerca de 50% das propriedades. A Seção 5.3 explica essa diferença.

Desafios e decisões na coleta. (i) O ambiente não acessava o Maven Central, o que exigiu compilar jqwik e PIT a partir do código-fonte (Seção 4.7). (ii) Por padrão, o jqwik reexecuta a semente que falhou na execução anterior; sob o PIT, isso faria um mutante influenciar o seguinte, e essa opção foi desligada. (iii) O StrykerJS para no primeiro teste que mata um mutante; para atribuir mortes a propriedades da mesma forma que o PIT, foi usado `disableBail`. (iv) Na análise, verificou-se que o grupo `DEFAULTS` do PIT 1.18 não inclui mais o operador que nega condições (`NegateConditionals`), substituído pela remoção de condicionais; isso muda a composição da categoria "Relacional" e é tratado na Seção 5.3.

Tabela 8. Resumo das 30 execuções por ecossistema (médias por execução; desvio-padrão entre parênteses quando diferente de zero).

Métrica	Java (jqwik + PIT)	Node.js (fast-check + StrykerJS)
Linhas de código do domínio / das propriedades e geradores	378 / 336	284 / 292
Propriedades implementadas	8	8
Mutantes gerados por execução	126	260 (1 com erro de execução)
Detectados (mortos + *timeout*)	63,0	135,2 (0,7)
Sobreviventes	56,0	100,8 (0,7)
Sem cobertura	7	23
Mutation score (%)	**50,00** (0,00)	**52,19** (0,27)
Mutation score nas categorias comuns (%)	48,65 (0,00)	57,18 (0,19)
Mutation score sem construtores e acessores (%)	48,42 (0,00)	52,81 (0,29)
Cobertura de linhas / de ramos do domínio (%)	91,07 / 73,26	91,35 / 81,81
Tempo da suíte de propriedades (s)	2,40 (0,41)	1,24 (0,09)
Tempo da análise de mutação (s)	34,04 (4,62)	28,88 (2,15)
Mutantes com resultado diferente entre execuções	0	4
Linha de base: *mutation score* só com os 11 cenários de exemplo (%)	92,06	94,62

5.2 Visualização dos resultados

A Figura 3 apresenta a distribuição do *mutation score* nas 30 execuções de cada ecossistema, em três recortes: todos os mutantes, apenas as categorias de operadores comuns às duas ferramentas e todos exceto os mutantes de construtores e métodos de acesso. Em todos os recortes, as 30 execuções do Java coincidem em um único valor, e as do Node.js variam menos de um ponto percentual, sempre acima do Java. A distância é pequena no recorte completo (2,2 pontos) e maior nas categorias comuns (8,5 pontos).

A Figura 4 mostra o *mutation score* por módulo da aplicação. Os dois ecossistemas têm o mesmo padrão: os módulos de cadastro (Eventos e Lotes) ficam entre 33% e 40%, e os módulos de Disponibilidade e Status, verificados por P5–P8, ficam acima de 60%. O módulo Status chega a 100% no Node.js e a 67% no Java; a diferença vem de quatro mutantes do PIT que trocam o valor de retorno dos métodos de transição por `null`, um tipo de mutação que o StrykerJS não aplica. O módulo Lotes tem exatamente o mesmo *mutation score* nos dois lados (33,3%).

A Figura 5 compara, para cada módulo, a proporção de mutantes cobertos pela suíte (círculo vazado) com a proporção detectada (círculo cheio). A distância entre os dois é o que a cobertura sozinha não revela. No Java, todos os módulos de regra exceto Eventos (80%) têm 100% dos mutantes cobertos, mas a detecção varia de 33% (Lotes, distância

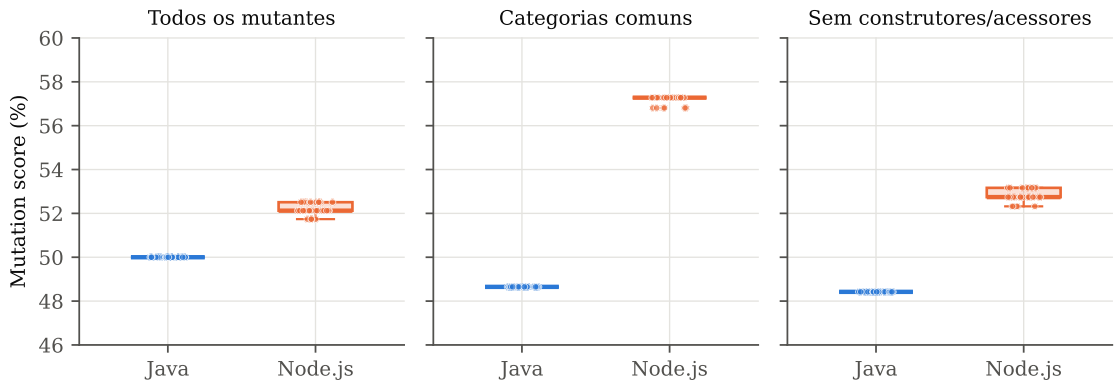


Figura 3. *Mutation score* por execução (30 por ecossistema), em três recortes de mutantes. Cada ponto é uma execução.

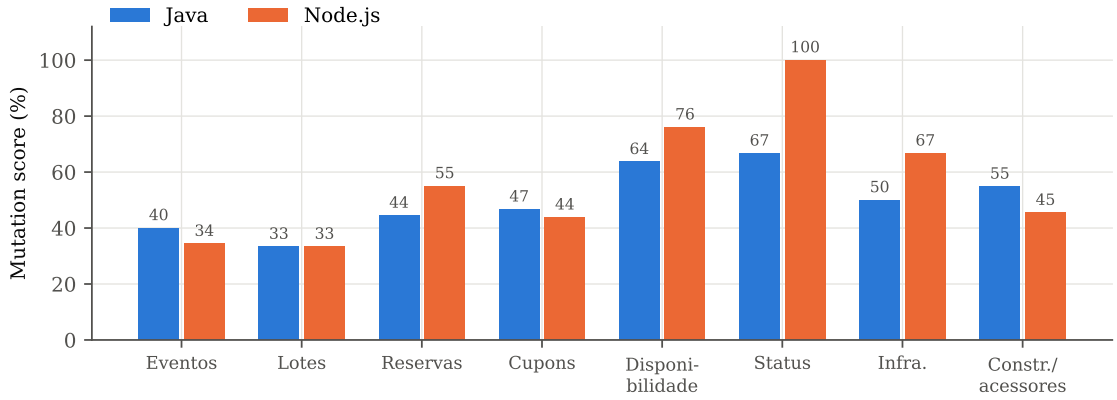


Figura 4. *Mutation score* por módulo da aplicação de referência (todas as execuções agregadas).

de 67 pontos) a 67% (Status). No Node.js, o padrão se repete, com a exceção do módulo Status, em que cobertura e detecção coincidem.

A Figura 6 decompõe os mutantes de cada execução por categoria de operador e por resultado. A composição é muito diferente: no Java, 41% dos mutantes são de "valor de retorno", categoria inexistente no StrykerJS; no Node.js, 16% são literais (principalmente textos de códigos de erro e nomes de status) e a categoria "Relacional" tem 47 mutantes por execução, contra 13 no Java. Os mutantes sem cobertura do Node.js (em cinza) estão quase todos dentro de ramos que lançam erros de validação, nunca executados pelas propriedades.

A Figura 7 mostra, para cada propriedade, a porcentagem dos mutantes detectados que ela mata (barra clara) e a porcentagem que só ela mata (barra escura). P4 (cupom inválido neutro) é a que mais mata nos dois ecossistemas, cerca de 56% a 59% dos detectados, mas quase sempre junto com P1 e P2. P8, a única propriedade que compara o sistema com um modelo de referência, é a que mais contribui com mortes exclusivas: 19% dos detectados no Java e 11% no Node.js. P1 não tem nenhuma morte exclusiva no Java: tudo o que ela detecta, P2 também detecta.

Por fim, a Figura 8 apresenta o tempo da análise de mutação. O PIT levou mais tempo (mediana de 33,7 s) que o StrykerJS (27,9 s), apesar de processar menos da metade

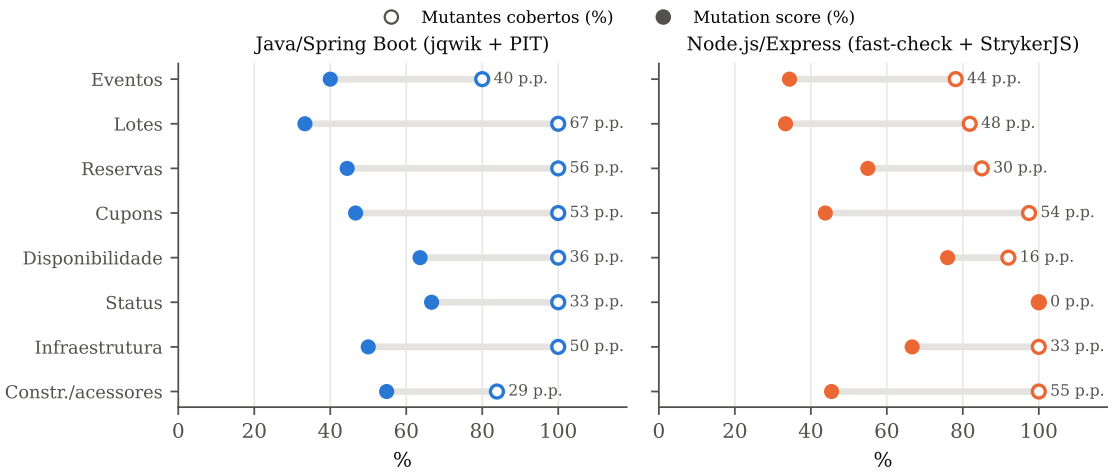


Figura 5. Mutantes cobertos pela suíte × *mutation score*, por módulo. O rótulo à direita é a diferença em pontos percentuais.

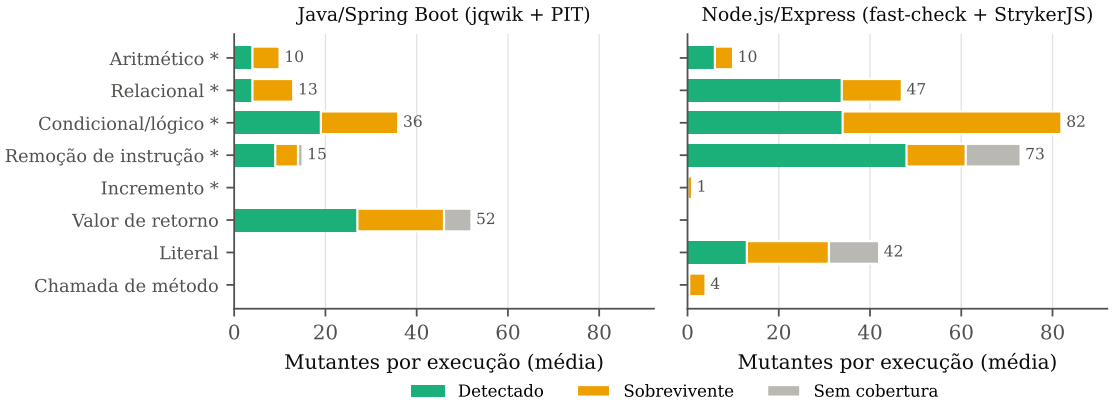


Figura 6. Mutantes por execução, por categoria de operador e resultado. Categorias marcadas com * existem nas duas ferramentas.

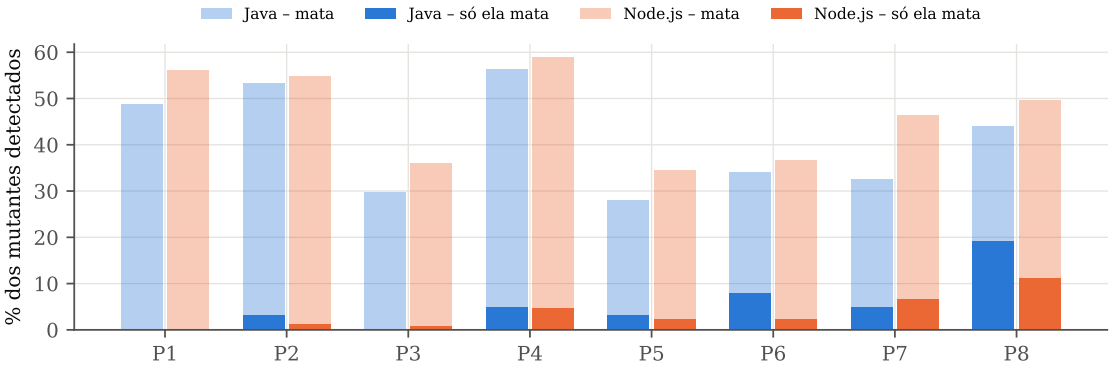


Figura 7. Porcentagem dos mutantes detectados que cada propriedade mata (clara) e que só ela mata (escura), média por execução.

dos mutantes: 0,27 s por mutante contra 0,11 s. O ponto isolado do Java (53,9 s) coincide com um período de atividade concorrente na máquina.

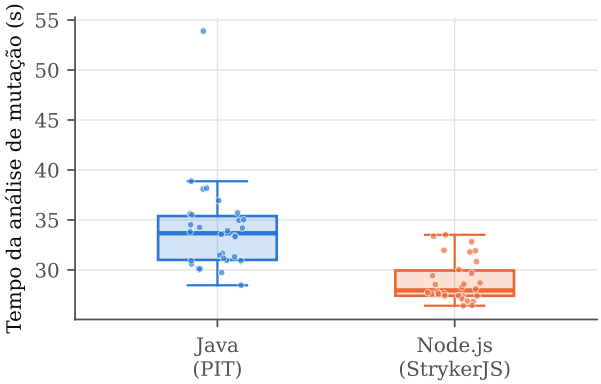


Figura 8. Tempo da análise de mutação por execução.

5.3 Discussão dos resultados

A discussão segue as questões de pesquisa da Tabela 2.

QP1 – Quais regras puderam ser expressas como propriedades equivalentes? As oito propriedades previstas foram implementadas nos dois ecossistemas com o mesmo enunciado, as mesmas faixas de geração e o mesmo número de tentativas (M2 = 8/8). Juntas, elas verificam 10 das 20 regras da especificação (M1 = 50%): as de preço, disponibilidade e status. As regras de validação de entrada (R1, R3, R4, R7, R9), de evento inativo (R2), de reserva vazia (R14), de aplicação de cupom (R10) e de entidade inexistente (R20) não têm propriedade correspondente, porque as propriedades do TCC I foram escolhidas a partir dos padrões mais frequentes na literatura [Corgozinho et al., 2023], que são de limite, relação e transição. A escrita foi equivalente em tamanho (336 e 292 linhas). A diferença de estilo é que o jqwik declara entradas por anotações (`@ForAll`, `@IntRange`) e combina geradores com `Combinators`, exigindo tipos explícitos, enquanto o fast-check compõe geradores em linha com `fc.record` e `fc.property`. Nenhuma propriedade precisou ser adaptada para caber em uma das bibliotecas.

QP2 – Qual o *mutation score* de cada suíte? 50,0% no Java e 52,2% no Node.js. Os dois valores são baixos quando comparados à linha de base de exemplos (92,1% e 94,6%). Isso não significa que exemplos são melhores que propriedades em geral, mas mostra o que cada suíte verifica: os cenários de exemplo foram escritos para conferir valores exatos e códigos de erro de todas as 20 regras, enquanto as propriedades verificam limites e relações de metade delas. O resultado é coerente com Ravi and Coblenz [2025], que encontraram grande variação de efetividade entre categorias de propriedades.

QP3 – A diferença é estatisticamente significativa, e de que tamanho? Sim, mas pequena em termos práticos. Em todas as 30 execuções o Node.js ficou acima do Java, o que dá $\hat{A}_{12} = 0$ e $p < 0{,}001$ (Seção 5.4). A diferença, porém, é de 2,2 pontos percentuais. Ela é "grande" no sentido estatístico porque as amostras quase não variam, não porque as suítes sejam muito diferentes. A ausência de variação também é um resultado: com 100 tentativas, as duas bibliotecas chegaram sempre aos mesmos mutantes, e o jqwik, que

avisou que os casos de borda excediam as tentativas, gerou entradas praticamente fixas.

QP4 – A diferença se mantém nas categorias comuns? Ela aumenta para 8,5 pontos (48,6% contra 57,2%; qui-quadrado sobre os mutantes, $\chi^2 = 48{,}1$, $p < 0{,}001$), mas a explicação está na composição das categorias, não nas suítes. No PIT 1.18, a categoria "Relacional" contém apenas mutantes de limite ($< \leftrightarrow <=$), que são difíceis de matar; no StrykerJS ela também inclui negações ($=== \rightarrow\ !==$), que qualquer propriedade que exercite o trecho mata. Comparando apenas os mutantes de limite, que existem nas duas ferramentas, os números se igualam: 13 mutantes por execução em cada lado, com 30,8% detectados no Java e 29,2% no Node.js. Quando o operador é de fato o mesmo, as duas suítes detectam a mesma coisa. Isso confirma, para o par PIT/StrykerJS, o que Kintis et al. [2018] e Monteiro et al. [2022] observaram entre ferramentas Java: o conjunto de operadores muda o *mutation score* mais do que a suíte avaliada.

QP5 – Os mutantes cobertos são também detectados? Não. A cobertura de linhas ficou acima de 91% nos dois ecossistemas e a de ramos em 73% (Java) e 82% (Node.js), enquanto o *mutation score* ficou em torno de 50%: cerca de 40 pontos de distância. No nível de módulo (Figura 5), o módulo Lotes tem todos os mutantes cobertos e apenas um terço detectado nos dois ecossistemas. A cobertura indicou corretamente onde a suíte não chega (os ramos de validação), mas não indicou que metade do código coberto não é verificado.

QP6 – Onde se concentram os sobreviventes, e por quê? A Tabela 9 classifica todos os mutantes não detectados em ao menos uma execução. Os equivalentes são raros (3% a 4%): um contador de identificadores decrescente, prefixos de identificador vazios e um atalho para cupom nulo, que não mudam o comportamento observável. O restante se divide em duas causas. A primeira é a **regra sem propriedade**: 51% dos não detectados no Java e 76% no Node.js estão em regras que nenhuma das oito propriedades verifica, principalmente validações de cupom (R9), de lote e capacidade (R3, R4) e de evento (R1, R2). O Node.js tem mais casos dessa classe porque o StrykerJS gera vários mutantes por condição composta e muta os textos dos códigos de erro. A segunda é a **propriedade fraca**: 46% no Java e 20% no Node.js. Aqui a regra é exercitada, mas o oráculo não percebe a mudança. O caso mais claro é o preço: um mutante que faz o subtotal valer sempre zero sobrevive a P1–P4 nos dois ecossistemas, porque total $= 0$ é não negativo, desconto $= 0$ está entre 0 e o subtotal, a ordem dos itens não muda zero e cupom inválido não altera zero. Nenhuma das quatro propriedades de preço fixa o valor esperado do subtotal ou do desconto.

A comparação entre propriedades reforça essa leitura. P8, que compara cada operação com um modelo da tabela de transições, é a que mais mata mutantes que nenhuma outra mata, e o módulo que ela verifica (Status) tem o maior *mutation score* nos dois ecossistemas. As propriedades de limite (P1, P2, P5) se sobrepõem quase por completo. Na terminologia de Bartocci et al. [2023], os mutantes de preço violam uma regra que P1–P4 deveriam proteger, mas não violam nenhuma das propriedades como foram escritas.

QP7 – Qual ecossistema exige mais tempo e esforço? O Node.js foi mais rápido nas duas medidas: a suíte rodou em 1,2 s contra 2,4 s (diferença dominada pela inicialização da JVM) e a mutação em 28,9 s contra 34,0 s, mesmo com o dobro de mutantes. Uma

Tabela 9. Classificação manual dos mutantes não detectados em ao menos uma execução.

Classe	Principais causas	Java	Node.js
Equivalente	Identificadores ainda únicos; busca de cupom nulo	2 (3%)	5 (4%)
Propriedade fraca	Desconto e subtotal só verificados por limites; retorno dos métodos não observado; recusa de confirmação sem verificar a condição	29 (46%)	25 (20%)
Regra sem propriedade	Validações de cupom, lote, evento e item; evento inativo; entidade inexistente; nomes de status	32 (51%)	97 (76%)
Total		63	127

explicação provável para o custo por mutante maior é a inicialização das JVMs auxiliares que o PIT usa para executar os mutantes isolados do processo principal. No esforço de configuração (M14, registrado em `docs/diario.md`), os dois caminhos padrão são parecidos em tamanho: dois arquivos de configuração e quatro artefatos no Java, quatro arquivos e quatro pacotes no Node.js. A diferença está nos detalhes não óbvios: no Java foi preciso declarar o *plugin* JUnit 5 do PIT (sem ele nenhum teste é encontrado) e desligar o reaproveitamento de sementes do jqwik; no Node.js não houve erro até a primeira execução, e o único ajuste foi metodológico (`disableBail`). Esse resultado é coerente com as barreiras de configuração relatadas por Sánchez et al. [2024].

5.4 Testes estatísticos

A Tabela 10 apresenta os testes de hipótese. Para o *mutation score*, nos três recortes, rejeita-se $H_{0,1}$: o Node.js teve *mutation score* maior em todas as comparações entre execuções ($\hat{A}_{12} = 0$, efeito grande). Para o tempo da análise de mutação, rejeita-se $H_{0,2}$: o Java foi mais lento em 90% das comparações ($\hat{A}_{12} = 0{,}90$, efeito grande). O teste de Shapiro-Wilk rejeitou a normalidade do *mutation score* do Node.js ($p < 0{,}001$) e não se aplica ao Java, cujo valor é constante, o que confirma a escolha de um teste não paramétrico. Como discutido na QP3 e na QP4, a significância estatística do *mutation score* decorre da baixa variância e da composição dos operadores; a diferença prática é de 2,2 pontos no recorte completo e desaparece quando se comparam apenas mutantes do mesmo tipo.

6 Ameaças à Validade

As ameaças foram organizadas pelas quatro validades de Wohlin et al. [2012]. A Tabela 11 resume cada uma, a mitigação adotada e o que poderia ser feito em trabalhos futuros; os parágrafos seguintes detalham as mais relevantes.

Validade de conclusão. No Java, as 30 execuções produziram exatamente o mesmo *mutation score*, o que reduz o teste de Mann-Whitney a uma comparação com empates em um dos grupos. A diferença, porém, não depende do teste: em todas as execuções o Node.js ficou acima do Java, o que o $\hat{A}_{12}$ igual a zero expressa diretamente, e o teste qui-quadrado

Tabela 10. Testes de Mann-Whitney U (bilateral, $n = 30$ por grupo) e tamanho de efeito $\hat{A}_{12}$ (probabilidade de Java > Node.js).

Métrica	Mediana Java	Mediana Node.js	U	p-valor	$\hat{A}_{12}$	Efeito
Mutation score (%)	50,00	52,12	0	$6{,}5 \times 10^{-13}$	0,00	grande
Mutation score, categorias comuns (%)	48,65	57,28	0	$1{,}5 \times 10^{-13}$	0,00	grande
Mutation score, sem acessores (%)	48,42	52,74	0	$6{,}5 \times 10^{-13}$	0,00	grande
Tempo da análise de mutação (s)	33,67	27,95	813	$8{,}4 \times 10^{-8}$	0,90	grande
Tempo da suíte de propriedades (s)	2,28	1,23	900	$3{,}0 \times 10^{-11}$	1,00	grande

sobre os mutantes das categorias comuns chega à mesma conclusão. O tempo foi medido em uma máquina virtual de 2 vCPUs; durante parte das primeiras execuções Java houve atividade concorrente leve na mesma máquina (escrita de código e instalação de pacotes), o que explica parte da dispersão do tempo Java e os poucos mutantes classificados como *timeout* nessas execuções.

Validade interna. A maior ameaça é a equivalência entre as implementações. Ela foi tratada com a especificação escrita antes do código e com os cenários compartilhados, mas há diferenças que são próprias de cada linguagem e que não podem ser eliminadas sem descaracterizar o ecossistema: o Java expõe dados por métodos de acesso (*getters*), que o PIT muta, enquanto o JavaScript usa campos diretos; os status do JavaScript são textos que o StrykerJS muta, enquanto os `enum` do Java não são mutados pelo PIT. A análise "sem construtores e acessores" e a análise por categorias comuns existem para isolar esses efeitos. As bibliotecas de PBT também diferem por dentro: o jqwik informou que os casos de borda excediam o número de tentativas, o que torna sua geração muito mais repetitiva que a do fast-check. Isso faz parte do fator estudado (o ecossistema), mas limita a atribuição das diferenças à linguagem em si.

Validade de construto. O *mutation score* mede a capacidade de detectar defeitos artificiais, não defeitos reais. Como as ferramentas geram conjuntos de mutantes diferentes [Kintis et al., 2018; Monteiro et al., 2022], o fator "ecossistema" mistura linguagem e ferramenta, e o estudo não separa os dois efeitos. A classificação dos sobreviventes foi feita por uma pessoa, mas as regras estão codificadas e versionadas no repositório, o que permite revisá-las.

Validade externa. A aplicação tem cerca de 300 linhas de domínio em cada linguagem e oito propriedades, e as propriedades verificam o domínio diretamente, sem passar pela camada HTTP. Os resultados não devem ser generalizados para APIs grandes, com persistência real ou com propriedades escritas em outro estilo (por exemplo, comparação com um modelo de referência completo).

Tabela 11. Ameaças à validade, mitigações adotadas e mitigações futuras.

Validade	Ameaça	Mitigação adotada	Mitigação futura
Conclusão	Variância nula do *mutation score* no Java, o que gera empates no teste de postos	Tamanho de efeito $\hat{A}_{12}$, teste qui-quadrado por mutante e estatísticas descritivas completas	Variar o número de tentativas e a estratégia de casos de borda
Conclusão	Tempo medido em máquina virtual compartilhada	30 execuções sequenciais, mediana e $\hat{A}_{12}$ em vez de média	Repetir em máquina dedicada
Interna	Implementações não equivalentes	Especificação única, 11 cenários compartilhados (125 passos) aprovados nos dois lados	Testes diferenciais gerados automaticamente entre as duas APIs
Interna	Propriedades mais fortes em um lado	Mesmos enunciados, geradores e tentativas, mapeados 1:1	Revisão das propriedades por um segundo pesquisador
Interna	Ferramentas Java compiladas do código-fonte	Mesmas *tags* das versões publicadas e `pom.xml` com as mesmas versões	Repetir via Maven com os binários publicados
Construto	*Mutation score* como medida de efetividade	Complementado por cobertura, análise por módulo e classificação manual	Usar defeitos reais injetados
Construto	Conjuntos de operadores diferentes entre PIT e StrykerJS	Análise restrita às cinco categorias comuns	Operadores equivalentes definidos manualmente
Construto	Mutantes equivalentes e classificação feita por uma pessoa	Regras de classificação explícitas e versionadas	Classificação independente com cálculo de concordância
Externa	Um domínio pequeno e oito propriedades	Domínio com regras numéricas, de estado e de disponibilidade	Replicar em APIs de outros domínios e tamanhos
Externa	Uma versão de cada ferramenta e propriedades no nível do domínio	Versões fixadas e documentadas	Incluir PBT via HTTP e outras bibliotecas

7 Conclusão

Este trabalho perguntou se suítes de testes baseadas em propriedades, derivadas das mesmas regras de negócio, têm a mesma efetividade em implementações equivalentes de uma API REST em Java/Spring Boot e em Node.js/Express. A resposta é: quase. Em 30 execuções por ecossistema, as oito propriedades detectaram 50,0% dos mutantes do PIT no Java e 52,2% dos mutantes do StrykerJS no Node.js. A diferença é estatisticamente significativa, porque o Node.js ficou acima em todas as execuções, mas tem apenas 2,2 pontos percentuais, e desaparece quando se comparam mutantes do mesmo tipo: nos mutantes de limite, presentes nas duas ferramentas, os dois ecossistemas detectaram 30,8% e 29,2%.

Os principais achados são:

1. **A ferramenta pesa mais que a suíte.** O StrykerJS gerou o dobro de mutantes do PIT, com composição diferente: o PIT muta valores de retorno, o StrykerJS muta literais e nega condições. As diferenças de *mutation score* entre ecossistemas, inclusive a maior (8,5 pontos nas categorias "comuns"), são explicadas pela composição dos operadores. Comparar *mutation score* entre ecossistemas só faz sentido no mesmo tipo de mutante.
2. **Cobertura alta não significou verificação.** As propriedades cobriram mais de 91% das linhas do domínio nos dois ecossistemas e detectaram cerca de metade dos mutantes; no módulo de lotes, todos os mutantes foram cobertos e um terço foi detectado.
3. **Os sobreviventes têm duas causas, não uma.** Poucos são equivalentes (3% a 4%). Metade ou mais estão em regras que nenhuma propriedade verifica (validações de entrada, evento inativo, entidade inexistente), e o restante sobrevive porque as propriedades de limite aceitam valores errados, como um subtotal sempre igual a zero.
4. **Propriedades com modelo de referência foram as mais efetivas.** P8, que compara cada transição de status com um modelo, foi a que mais matou mutantes exclusivos, e o módulo que ela verifica teve o maior *mutation score* nos dois ecossistemas. Propriedades de limite se sobrepuseram quase por completo.
5. **O resultado do PBT quase não variou entre execuções.** O *mutation score* do Java foi idêntico nas 30 execuções; no Node.js, apenas 4 de 260 mutantes mudaram de resultado.
6. **Node.js foi mais rápido e mais simples de configurar.** A mutação levou 28,9 s contra 34,0 s, com o dobro de mutantes, e a configuração não teve armadilhas; no Java, foi preciso conhecer o *plugin* JUnit 5 do PIT e o reaproveitamento de sementes do jqwik.

Para quem testa APIs com PBT, a implicação prática é que propriedades de limite e de neutralidade, as mais comuns na prática [Corgozinho et al., 2023], protegem pouco os cálculos: é preciso combiná-las com oráculos de valor (um modelo de referência do preço, por exemplo) e cobrir também as regras de validação. Para quem compara ferramentas ou

ecossistemas, a implicação é reportar o *mutation score* por categoria de operador, e não apenas o total.

Como contribuições, o trabalho entrega um experimento comparativo com teste estatístico, a classificação dos mutantes sobreviventes por regra e por propriedade e um repositório reprodutível com a especificação, as duas APIs, as suítes, os scripts e os dados brutos das 60 execuções.

Trabalhos futuros. (i) Fortalecer as propriedades de preço com um modelo de referência e cobrir as regras de validação, medindo quanto o *mutation score* sobe em cada ecossistema; (ii) repetir o experimento com o grupo `STRONGER` do PIT e com o conjunto de operadores do StrykerJS restrito aos equivalentes do PIT; (iii) variar o número de tentativas e o modo de casos de borda do jqwik, para entender a falta de variação observada; (iv) aplicar PBT com estado diretamente sobre a camada HTTP; e (v) replicar o estudo em outros domínios, em TypeScript e com propriedades geradas automaticamente.

Referências

Bernhard K. Aichernig and Richard Schumi. Property-based testing of web services by deriving properties from business-rule models. *Software and Systems Modeling*, 18(2): 889–911, 2019.

Andrea Arcuri and Lionel Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE '11)*, pages 1–10. ACM, 2011.

Ezio Bartocci, Leonardo Mariani, Dejan Ničković, and Drishti Yadav. Property-based mutation testing. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 222–233. IEEE, 2023.

Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In *Encyclopedia of Software Engineering*. Wiley, 1994.

Moritz Beller, Chu-Pan Wong, Johannes Bader, Andrew Scott, Mateusz Machalica, Satish Chandra, and Erik Meijer. What it would take to use mutation testing in industry: A study at Facebook. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 268–277. IEEE, 2021.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279. ACM, 2000.

Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. PIT: A practical mutation testing tool for Java. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA 2016)*, pages 449–452. ACM, 2016.

Arthur Lisboa Corgozinho, Marco Tulio Valente, and Henrique Rocha. How developers implement property-based tests. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 380–384. IEEE, 2023.

Nicolas Dubien. fast-check: Property based testing for JavaScript and TypeScript. https://fast-check.dev, 2026. Acesso em: 15 out. 2026.

Harrison Goldstein, Joseph W. Cutler, Daniel Dickstein, Benjamin C. Pierce, and Andrew Head. Property-based testing in practice. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*, pages 1–13. ACM, 2024.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

jqwik team. jqwik user guide. https://jqwik.net/docs/current/user-guide.html, 2026. Acesso em: 15 out. 2026.

Marinos Kintis, Mike Papadakis, Andreas Papadopoulos, Evangelos Valvis, Nicos Malevris, and Yves Le Traon. How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering*, 2018.

Thomas Laurent, Mike Papadakis, Marinos Kintis, Christopher Henard, Yves Le Traon, and Anthony Ventresque. Assessing and improving the mutation testing practice of PIT. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 430–435. IEEE, 2017.

Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1): 50–60, 1947.

Ricardo Monteiro, Vinicius Humberto Serapilha Durelli, Marcelo Eler, and Andre Endo. An empirical analysis of two mutation testing tools for Java. In *Anais do VII Simpósio Brasileiro de Testes de Software Sistemático e Automatizado (SAST)*, pages 49–58, Porto Alegre, 2022. SBC.

Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

Savitha Ravi and Michael Coblenz. An empirical evaluation of property-based testing in Python. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2), 2025. doi: 10.1145/3764068.

Ana B. Sánchez, José A. Parejo, Sergio Segura, Amador Durán, and Mike Papadakis. Mutation testing in practice: Insights from open-source software developers. *IEEE Transactions on Software Engineering*, 50(5):1130–1143, 2024.

Stryker Mutator. StrykerJS documentation. https://stryker-mutator.io/docs/stryker-js/introduction/, 2026. Acesso em: 15 out. 2026.

Ticketmaster Developer. Discovery API v2.0 documentation. https://developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2/, 2026. Acesso em: 03 jun. 2026.

András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.